Object-Oriented Principles in PHP (2024 Edition) — Study Notes

Course: Laracasts — Object-Oriented Principles in PHP (2024 Edition) **Student:** Benjamin **Stack target:** PHP 8.3 / 8.4 (Laravel / Filament / Inertia)

Recurring Themes (read this first)

These are the ideas the whole course keeps hammering. Every episode is a different angle on these.

- **Program to contracts, not concretes.** Depend on interfaces, not specific classes.
- **Inject dependencies.** Don't `new` things inside a class — pass them in.
- **Keep classes small and focused.** One reason to change.
- **Lean on the type system.** Scalar types, union types, readonly, enums. Let PHP catch your bugs.
- **Composition over inheritance.** If you can't say "X *is a* Y," don't use `extends`.
- **Let modern PHP kill boilerplate.** `readonly`, enums, property hooks, asymmetric visibility.

Episode 1 — Classes

SUMMARY

A **class** is a blueprint. An **object** is an actual instance made from that blueprint. A class defines properties (data) and methods (behavior), and the constructor (`__construct`) runs when you `new` it up. Before classes, you'd pass associative arrays around — classes replace that with something typed, expressive, and self-documenting.

KEY CODE (PHP 8.3)

```php
<?php

class Invoice
{
    public function __construct(
        public string $customer,
        public float $amount,
    ) {}

    public function format(): string
    {
        return "{$this->customer} owes R{$this->amount}";
    }
}

$invoice = new Invoice('Benjamin', 2500.00);
echo $invoice->format(); // Benjamin owes R2500
```

PRINCIPLES

- Constructor property promotion (PHP 8+) removes boilerplate — no need to declare the property *and* assign it in the constructor.
- A class is a **type**. `Invoice` in a type hint is infinitely more useful than `array`.

EXERCISE

Build a `Course` class with `title`, `instructor`, `episodeCount`, and a `describe()` method. Instantiate two courses and echo their descriptions.

Episode 2 — Objects

SUMMARY

Objects are **passed by reference**, not by value. That means when you hand an object to a function and the function mutates it, the original changes too. Objects also have **identity** (`===` checks if two variables point to the *same* instance) distinct from **equality** (`==` checks if two instances have the same property values). This is why objects beat associative arrays: they carry behavior, types, and identity.

KEY CODE

```
$a = new Invoice('Ben', 100);
$b = $a;           // same object, not a copy
$b->amount = 500;
echo $a->amount;   // 500 — both variables point to the SAME object

$c = new Invoice('Ben', 500);
var_dump($a == $c);  // true  — same values
var_dump($a === $c); // false — different instances
```

If you want an actual copy: `$b = clone $a;`.

PRINCIPLES

- Reference semantics is the #1 source of "wait, why did that change?" bugs. Internalize it.
- `array` is dumb data. Objects are smart data. Reach for objects.

EXERCISE

Create two `User` objects with the same name and email. Use `==` and `===` on them and predict the output before running it.

Episode 3 — DTOs, Types, and Static Analysis

SUMMARY

A **DTO (Data Transfer Object)** is a plain, immutable object whose only job is to carry structured data between layers of your app. No behavior, no logic — just typed properties. PHP's type system (scalar types, union types, nullable types, `readonly`, enums) makes DTOs bulletproof. Tools like **PHPStan** or **Psalm** then catch type errors at static-analysis time, before the code even runs.

KEY CODE (PHP 8.2+ — `READONLY` CLASS)

```php
<?php

readonly class CreateUserDTO
{
    public function __construct(
        public string $name,
        public string $email,
        public Role $role,   // enum, see below
        public ?int $age = null,
    ) {}
}

enum Role: string
{
    case Admin = 'admin';
    case Editor = 'editor';
    case Viewer = 'viewer';
}

$dto = new CreateUserDTO(
    name: 'Benjamin',
    email: 'ben@stratusolve.co.za',
    role: Role::Admin,
);

// $dto->name = 'Other';  // TypeError — readonly class, can't mutate
```

PRINCIPLES

- **Readonly** + **enums** + **typed properties** = compiler-enforced correctness.
- If a function takes a DTO, you know exactly what's in it. No `$data['email'] ?? null` guessing.
- Static analysis (PHPStan level 8) is free bugs-caught-for-you. Turn it on.

TIES BACK TO

Episode 1 (classes as types) taken to its logical extreme.

EXERCISE

Convert `['title' => 'PHP OOP', 'instructor' => 'Jeffrey', 'hours' => 8]` into a `readonly` `CourseDTO` with a `Difficulty` enum (`Beginner`, `Intermediate`, `Advanced`).

Episode 4 — Dependencies, Coupling, and Interfaces

SUMMARY

A **dependency** is anything your class needs to do its job. A **hard dependency** is when you `new` that thing up inside the class — now your class is glued to that specific implementation and you can't swap it or mock it. The fix is **dependency injection via the constructor**: declare what you need as a

parameter, let the caller provide it. And don't ask for a concrete class — ask for an **interface** (a contract).

KEY CODE

```php
<?php

interface Mailer
{
    public function send(string $to, string $body): void;
}

class SmtpMailer implements Mailer
{
    public function send(string $to, string $body): void { /* ... */ }
}

class LogMailer implements Mailer
{
    public function send(string $to, string $body): void {
        error_log("Pretending to email {$to}: {$body}");
    }
}

class OrderNotifier
{
    public function __construct(private Mailer $mailer) {}  // ← depends on the CONTRACT

    public function notify(Order $order): void
    {
        $this->mailer->send($order->email, "Your order is confirmed.");
    }
}

// Prod:
$notifier = new OrderNotifier(new SmtpMailer());
// Tests:
$notifier = new OrderNotifier(new LogMailer());
```

PRINCIPLES

- "New is glue." Every `new` inside a class hard-couples it to a concrete type.
- Constructor injection + interface type-hint = swap any implementation without touching the class.
- This is the foundation Laravel's service container is built on.

EXERCISE

Take a class that does `$client = new \GuzzleHttp\Client();` inside a method and refactor it to accept a `HttpClient` interface in the constructor.

Episode 5 — Inheritance and Abstract Classes

SUMMARY

`extends` lets one class inherit properties and methods from another. You can override methods and call the parent's version with `parent::method()`. An **abstract class** is a class that can't be instantiated directly — it exists only to be extended, and it can declare **abstract methods** that subclasses *must* implement. Inheritance is a strong is-a relationship. Overuse it and you get brittle hierarchies. The course's rule of thumb: **favor composition over inheritance**.

KEY CODE

```php
<?php
abstract class Report
{
    abstract public function data(): array;

    public function render(): string
    {
        return json_encode($this->data());  // shared behavior
    }
}

class SalesReport extends Report
{
    public function data(): array
    {
        return ['revenue' => 12_500, 'orders' => 48];
    }
}

echo (new SalesReport())->render();
```

PRINCIPLES

- Inheritance models "is a." Composition models "has a." Ask which one fits before reaching for `extends`.
- Deep inheritance trees are a smell. If you've got three levels, something's probably wrong.
- Abstract classes are useful when you have **genuinely shared behavior** plus **required hooks** subclasses fill in.

TIES BACK TO

Episode 4 — an abstract class is often a heavier alternative to an interface. Interfaces are usually the better default.

EXERCISE

Design a `PaymentGateway` abstract class with a concrete `charge()` method that calls an abstract `authorize()` method. Implement `StripeGateway` and `PaypalGateway`.

Episode 6 — Interfaces as Feature Filters

SUMMARY

Beyond contracts, interfaces are **capability markers**. Instead of checking "is this thing a `Product`?" you check "does this thing implement `Discountable`?" This lets collaborators branch or filter on *what an object can do*, not on what it is. Think `Countable`, `Iterator`, `JsonSerializable` in the standard library — same idea.

KEY CODE

```php
<?php
interface Cacheable
```

```php
{
    public function cacheKey(): string;
    public function cacheTtl(): int;
}

class Product implements Cacheable
{
    public function __construct(public int $id, public string $name) {}

    public function cacheKey(): string { return "product:{$this->id}"; }
    public function cacheTtl(): int { return 3600; }
}

class Cache
{
    public function remember(object $item, callable $producer): mixed
    {
        if (! $item instanceof Cacheable) {
            return $producer();  // skip caching
        }

        // use $item->cacheKey() + $item->cacheTtl() ...
    }
}
```

PRINCIPLES

- Interfaces are cheap — create one for every distinct capability.
- `instanceof SomeInterface` is usually better than `instanceof SomeConcreteClass`.
- Small, focused interfaces (ISP — Interface Segregation Principle) beat one giant "god interface."

TIES BACK TO

Episode 4 established "code to an interface." Episode 6 is *using* that idea as a dispatch mechanism.

EXERCISE

Create a `Sortable` interface with `sortKey(): int`. Implement it on two different classes. Write a function that sorts a mixed array of any objects that implement `Sortable`.

Episode 7 — Encapsulation and Visibility

SUMMARY

Encapsulation is hiding internal state behind intentional methods. `public`, `protected`, `private` are your visibility tools. The goal isn't secrecy — it's **controlling the surface area** so callers can't put your object into an invalid state. PHP 8.4 added **asymmetric visibility** (`public private(set)`) so you can have a property that's publicly *readable* but only privately *writable* — removing the need for a getter in many cases.

KEY CODE (PHP 8.4)

```php
<?php

class BankAccount
{
    public function __construct(
        public private(set) float $balance = 0.0,   // readable anywhere, writable only inside th
    ) {}

    public function deposit(float $amount): void
```

```
    {
        if ($amount <= 0) {
            throw new InvalidArgumentException('Must be positive');
        }
        $this->balance += $amount;
    }

    public function withdraw(float $amount): void
    {
        if ($amount > $this->balance) {
            throw new RuntimeException('Insufficient funds');
        }
        $this->balance -= $amount;
    }
}

$acct = new BankAccount();
$acct->deposit(100);
echo $acct->balance;      // 100 — reading is fine
// $acct->balance = -1;  // Error — can't write from outside
```

PRINCIPLES

- Make properties as restrictive as you can get away with. Widen later if you need to.
- Encapsulation isn't about hiding info from humans — it's about protecting invariants.
- Asymmetric visibility kills the "write a getter for every property" anti-pattern.

EXERCISE

Write an `Order` class that guarantees its `status` property is always one of `pending`, `paid`, `shipped`, `cancelled`, and can only transition in a valid order. Use asymmetric visibility + an enum.

Episode 8 — From Getters and Setters to Property Hooks

SUMMARY

Naive getters and setters (`getName()` , `setName($x)`) leak encapsulation — they're just a thin varnish over public properties and they bloat the class. PHP 8.4's **property hooks** let you intercept reads and writes on a property directly, so you keep the natural `->name` syntax and still get validation, derivation, or transformation.

KEY CODE (PHP 8.4)

```
<?php

class User
{
    public string $email {
        set (string $value) {
            if (! filter_var($value, FILTER_VALIDATE_EMAIL)) {
                throw new InvalidArgumentException('Invalid email');
            }
            $this->email = strtolower(trim($value));
        }
    }

    // Derived / virtual property — no storage, just a getter hook
    public string $firstName {
        get => explode(' ', $this->name)[0] ?? '';
    }
```

```
    public function __construct(public string $name, string $email)
    {
        $this->email = $email;   // runs the set hook
    }
}

$u = new User('Benjamin Shaw', 'BEN@STRATUSOLVE.CO.ZA');
echo $u->email;      // ben@stratusolve.co.za
echo $u->firstName;  // Benjamin
```

PRINCIPLES

- If a getter just `return $this->x`, you probably want a public (or `public private(set)`) property.
- Use hooks for validation, normalization, or derived values — anywhere the value needs more than plain storage.
- Fewer methods = clearer API.

TIES BACK TO

Episode 7 — hooks are encapsulation *without* the boilerplate tax.

EXERCISE

Refactor a class with `getPrice()`, `setPrice()`, and `getFormattedPrice()` into one that exposes `price` (with a set hook validating `> 0`) and `formattedPrice` (with a get hook returning `"R1,234.56"`).

Episode 9 — Understanding Object Composition and Abstractions

SUMMARY

Composition means building complex behavior by plugging small, focused objects together. Each object does one thing well; the whole is assembled at construction time. The **Dependency Inversion Principle** says both high-level and low-level code should depend on abstractions (interfaces), not on each other. The warning: don't abstract *too early*. A **leaky abstraction** is an interface that exposes its underlying implementation (e.g., a `FileStore` interface with a method called `ftpUpload()`).

KEY CODE

```
<?php

interface PriceCalculator { public function total(array $items): float; }
interface TaxPolicy { public function apply(float $subtotal): float; }
interface DiscountPolicy { public function apply(float $subtotal): float; }

class Checkout implements PriceCalculator
{
    public function __construct(
        private TaxPolicy $tax,
        private DiscountPolicy $discount,
    ) {}

    public function total(array $items): float
    {
        $subtotal = array_sum(array_column($items, 'price'));
        $afterDiscount = $this->discount->apply($subtotal);
        return $this->tax->apply($afterDiscount);
    }
```

```
}
```

Each policy is its own tiny class. Swap the discount from "10% off" to "flat R50 off" without touching `Checkout`.

PRINCIPLES

- Small pieces, well named, plugged together.
- The right abstraction is the one that doesn't lie about its underlying implementation.
- Don't create an interface for something that has exactly one implementation and no reason to grow. YAGNI.

TIES BACK TO

Episodes 4, 5, 6 all converge here. This is the synthesis episode.

EXERCISE

Model a `NotificationSender` that composes a `Channel` (email, SMS, Slack) and a `Formatter` (plain, markdown, html). Compose, don't extend.

Episode 10 — Object-Oriented Workshop (Capstone)

SUMMARY

The final project: build a small **file-storage API** with two interchangeable drivers — a local filesystem one and an Amazon S3 one — both implementing the same `Storage` interface. You start procedurally (functions, arrays, hard-coded paths) and incrementally refactor into a clean OO design, applying every principle from the course.

KEY CODE (THE CONTRACT + TWO IMPLEMENTATIONS)

```php
<?php

interface Storage
{
    public function put(string $path, string $contents): void;
    public function get(string $path): string;
    public function delete(string $path): void;
    public function exists(string $path): bool;
}

final class LocalStorage implements Storage
{
    public function __construct(private readonly string $root) {}

    public function put(string $path, string $contents): void
    {
        $full = $this->root . DIRECTORY_SEPARATOR . ltrim($path, '/');
        @mkdir(dirname($full), recursive: true);
        file_put_contents($full, $contents);
    }

    public function get(string $path): string
    {
        return file_get_contents($this->root . '/' . ltrim($path, '/'))
            ?: throw new RuntimeException("Missing: {$path}");
    }
```

```php
    public function delete(string $path): void
    {
        @unlink($this->root . '/' . ltrim($path, '/'));
    }

    public function exists(string $path): bool
    {
        return file_exists($this->root . '/' . ltrim($path, '/'));
    }
}

final class S3Storage implements Storage
{
    public function __construct(
        private readonly S3Client $client,
        private readonly string $bucket,
    ) {}

    public function put(string $path, string $contents): void
    {
        $this->client->putObject([
            'Bucket' => $this->bucket,
            'Key' => $path,
            'Body' => $contents,
        ]);
    }

    public function get(string $path): string { /* getObject */ }
    public function delete(string $path): void { /* deleteObject */ }
    public function exists(string $path): bool { /* doesObjectExist */ }
}

// Anything consuming Storage doesn't know or care which driver it got
class BackupService
{
    public function __construct(private Storage $storage) {}

    public function backup(string $name, string $data): void
    {
        $this->storage->put("backups/{$name}.txt", $data);
    }
}
```

PRINCIPLES APPLIED (WHICH EPISODE EACH CAME FROM)

- `Storage` interface → **Ep 4 / Ep 6** (program to contracts, interfaces as capability).
- Constructor injection of `Storage` and `S3Client` → **Ep 4** (DI).
- `readonly` properties → **Ep 3** (types).
- `final class` → **Ep 5** (disallow inheritance when it's not designed for it — prefer composition).
- `BackupService` composes `Storage`, doesn't extend anything → **Ep 9** (composition).
- Private state, public behavior only → **Ep 7** (encapsulation).

EXERCISE

Add a third driver — `InMemoryStorage` — for testing. Use it in a unit test for `BackupService`. You should not have to touch `BackupService` at all.

Cheat Sheet — PHP 8.3 / 8.4 Features to Lean On

Feature	When to use	Episode
Constructor property promotion	Always, instead of declaring + assigning	1
`readonly` property / class	DTOs, value objects, anything immutable	3
Enums (backed)	Finite sets: statuses, roles, types	3

Typed properties + return types	Everywhere. Always.	3
`private(set)` asymmetric visibility	Public-read, private-write properties	7
Property hooks (`get` / `set`)	Replace most getters/setters	8
`final class`	Close the door on unintended extension	5, 10

Study Workflow

1. Watch / re-watch the episode.
2. Come back here and say **"quiz me on OOP Episode X"** — get a code challenge.
3. Write the answer from scratch (no peeking).
4. Paste it back for feedback.
5. Move to the next episode only when you can explain the principle in your own words.

Red flag: if you find yourself writing `new SomeClass()` inside another class, or typing a method parameter as a concrete class, stop and ask — should this be injected? Should this be an interface?